

MCA Semester 1	Subject : Advanced Data Structures Lab
Name : Mukund Gangurde	Topic: Binary Search Trees
Roll No. : MCA2511	Date : 15-12-2025

Binary Search Tree

Code:

```
import java.util.Scanner;

//Node Template
class Node
{
    int data;
    Node left;
    Node right;

    //Parameterized Constructor
    public Node(int d)
    {
        data = d;
        left = null;
        right = null;
    }
}

//end of Node

//Tree Template
class BSTree
{
    Node root;
    int cnt;

    //Default Constructor
    public BSTree()
    {
        root = null;
        cnt = 0;
    }

    //Insert
    public void Insert(int x)
    {
        //Make a new node
        Node t = new Node(x);
```

```
//First node in the BST
if(root == null)
{
    root = t;
    cnt++;
    return;
}

//Traverse
Node prev = null;
Node tmp = root;

while (tmp!=null)
{
    prev=tmp;

    if(t.data < tmp.data)
    {
        tmp = tmp.left;
    }
    else if(t.data > tmp.data)
    {
        tmp = tmp.right;
    }
    else
    {
        System.out.println("Duplicate!");
        return;
    }
}

} //end of while

//Attach t to parent node
if(t.data<prev.data)
{
    prev.left = t;
}
else
{
    prev.right = t;
}
cnt++;
} //end of Insert

//Display
public void Display()
```

```
{
    System.out.print("Preorder Traversal: ");
    Preorder(root);
    System.out.println();

    System.out.print("Inorder Traversal: ");
    Inorder(root);
    System.out.println();

    System.out.print("Postorder Traversal: ");
    Postorder(root);
    System.out.println();
} //end of Display

//Preorder
void Preorder(Node r)
{
    if(r!=null)
    {
        System.out.print(r.data + " ");
        Preorder(r.left);
        Preorder(r.right);
    }
} //end of Preorder

//Inorder
void Inorder(Node r)
{
    if(r!=null)
    {
        Inorder(r.left);
        System.out.print(r.data + " ");
        Inorder(r.right);
    }
} //end of Inorder

//Postorder
void Postorder(Node r)
{
    if(r!=null)
    {
        Postorder(r.left);
        Postorder(r.right);
        System.out.print(r.data + " ");
    }
} //end of Postorder
```

```
// Count
public void Count()
{
    System.out.println("Total nodes in BST: " + cnt);
}
```

```
//Search
public void Search(int x)
{
    boolean flag = false;
    Node tmp = root;

    while(tmp!=null)
    {
        if(x<tmp.data)
        {
            tmp = tmp.left;
        }
        else if(x>tmp.data)
        {
            tmp = tmp.right;
        }
        else
        {
            flag = true;
            break;
        }
    }

    if(flag==true)
    {
        System.out.println(x + " is found");
    }
    else
    {
        System.out.println(x + " is not found");
    }
}
} //end of Search
```

```
//FindMax
public void FindMax()
{
    if(root==null)
    {
        System.out.println("Empty BST!");
    }
}
```

```
        return;
    }
    Node tmp = root;
    while(tmp.right != null)
    {
        tmp = tmp.right;
    }

    System.out.println("Maximum node: "+ tmp.data);
} //end of FindMax

//FindMin
public void FindMin()
{
    if(root==null)
    {
        System.out.println("Empty BST!");
        return;
    }
    Node tmp = root;
    while(tmp.left != null)
    {
        tmp = tmp.left;
    }

    System.out.println("Minimum node: "+ tmp.data);
} //end of FindMin

} //end of BsearchTree

class BST
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        BSTree t = new BSTree();
        int ch,x;
        do
        {
            System.out.println("\n-----");
            System.out.println("-----Binary Search Tree-----");
            System.out.println("-----");
            System.out.println("1. Insert a Value");
            System.out.println("2. Display the BST");
            System.out.println("3. Search for a node in the BST");
            System.out.println("4. Count the number of nodes in the BST");
```

```
System.out.println("5. Find Max");
System.out.println("6. Find Min");
System.out.println("7. Exit\n");

System.out.print("Enter Your Choice: ");
ch = sc.nextInt();

switch(ch)
{
    case 1:
        System.out.print("Enter Data Value: ");
        x = sc.nextInt();
        t.Insert(x);
        break;

    case 2:
        t.Display();
        break;

    case 3:
        System.out.print("Enter Data Value to Search: ");
        x = sc.nextInt();
        t.Search(x);
        break;

    case 4:
        t.Count();
        break;

    case 5:
        t.FindMax();
        break;

    case 6:
        t.FindMin();
        break;

    case 7:
        System.out.println("Exiting .... ");
        break;

    default:
        System.out.println("Incorrect Choice Try Again.... ");
        break;
}
```

```
        } while(ch!=7);  
    } //end of psvm  
} //end of BST
```

Output:

```
A:\MCA2511\DS_LAB>javac 22BST.java
```

```
A:\MCA2511\DS_LAB>java BST
```

```
-----Binary Search Tree-----
```

1. Insert a Value
2. Display the BST
3. Search for a node in the BST
4. Count the number of nodes in the BST
5. Find Max
6. Find Min
7. Exit

```
Enter Your Choice: 1
```

```
Enter Data Value: 50
```

```
-----Binary Search Tree-----
```

1. Insert a Value
2. Display the BST
3. Search for a node in the BST
4. Count the number of nodes in the BST
5. Find Max
6. Find Min
7. Exit

```
Enter Your Choice: 1
```

```
Enter Data Value: 45
```

```
-----Binary Search Tree-----
```

1. Insert a Value
2. Display the BST
3. Search for a node in the BST
4. Count the number of nodes in the BST
5. Find Max
6. Find Min
7. Exit

```
Enter Your Choice: 1
```

```
Enter Data Value: 60
```

-----Binary Search Tree-----

1. Insert a Value
2. Display the BST
3. Search for a node in the BST
4. Count the number of nodes in the BST
5. Find Max
6. Find Min
7. Exit

Enter Your Choice: 1
Enter Data Value: 100

-----Binary Search Tree-----

1. Insert a Value
2. Display the BST
3. Search for a node in the BST
4. Count the number of nodes in the BST
5. Find Max
6. Find Min
7. Exit

Enter Your Choice: 1
Enter Data Value: 40

-----Binary Search Tree-----

1. Insert a Value
2. Display the BST
3. Search for a node in the BST
4. Count the number of nodes in the BST
5. Find Max
6. Find Min
7. Exit

Enter Your Choice: 1
Enter Data Value: 42

-----Binary Search Tree-----

1. Insert a Value
2. Display the BST
3. Search for a node in the BST
4. Count the number of nodes in the BST
5. Find Max
6. Find Min
7. Exit

Enter Your Choice: 1
Enter Data Value: 8

-----Binary Search Tree-----

1. Insert a Value
2. Display the BST
3. Search for a node in the BST
4. Count the number of nodes in the BST
5. Find Max
6. Find Min
7. Exit

Enter Your Choice: 1
Enter Data Value: 80

-----Binary Search Tree-----

1. Insert a Value
2. Display the BST
3. Search for a node in the BST
4. Count the number of nodes in the BST
5. Find Max
6. Find Min
7. Exit

Enter Your Choice: 2
Preorder Traversal: 50 45 40 8 42 60 100 80
Inorder Traversal: 8 40 42 45 50 60 80 100
Postorder Traversal: 8 42 40 45 80 100 60 50

-----Binary Search Tree-----

1. Insert a Value
2. Display the BST
3. Search for a node in the BST
4. Count the number of nodes in the BST
5. Find Max
6. Find Min
7. Exit

Enter Your Choice: 3
Enter Data Value to Search: 50
50 is found

-----Binary Search Tree-----

1. Insert a Value
2. Display the BST
3. Search for a node in the BST
4. Count the number of nodes in the BST
5. Find Max
6. Find Min
7. Exit

Enter Your Choice: 3
Enter Data Value to Search: 1
1 is not found

-----Binary Search Tree-----

1. Insert a Value
2. Display the BST
3. Search for a node in the BST
4. Count the number of nodes in the BST
5. Find Max
6. Find Min
7. Exit

Enter Your Choice: 4
Total nodes in BST: 8

-----Binary Search Tree-----

1. Insert a Value
2. Display the BST
3. Search for a node in the BST
4. Count the number of nodes in the BST
5. Find Max
6. Find Min
7. Exit

Enter Your Choice: 5
Maximum node: 100

-----Binary Search Tree-----

1. Insert a Value
2. Display the BST
3. Search for a node in the BST
4. Count the number of nodes in the BST
5. Find Max
6. Find Min
7. Exit

Enter Your Choice: 6
Minimum node: 8

-----Binary Search Tree-----

1. Insert a Value
2. Display the BST
3. Search for a node in the BST
4. Count the number of nodes in the BST
5. Find Max
6. Find Min
7. Exit

Enter Your Choice: 7
Exiting

A:\MCA2511\DS_LAB>